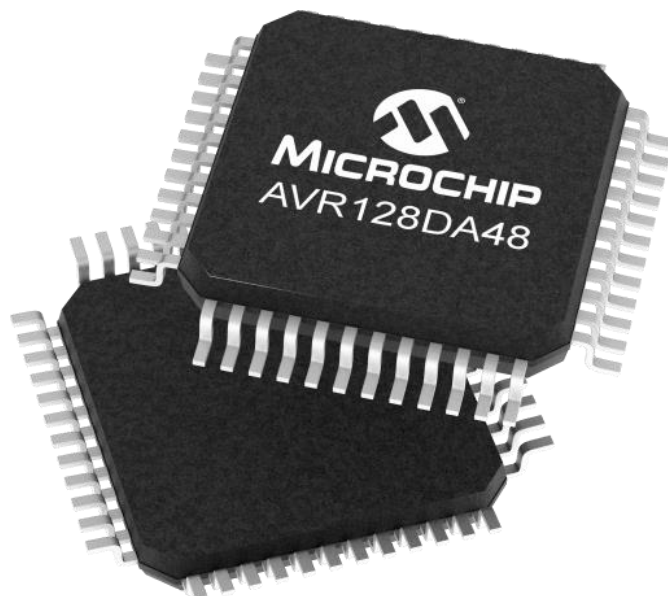
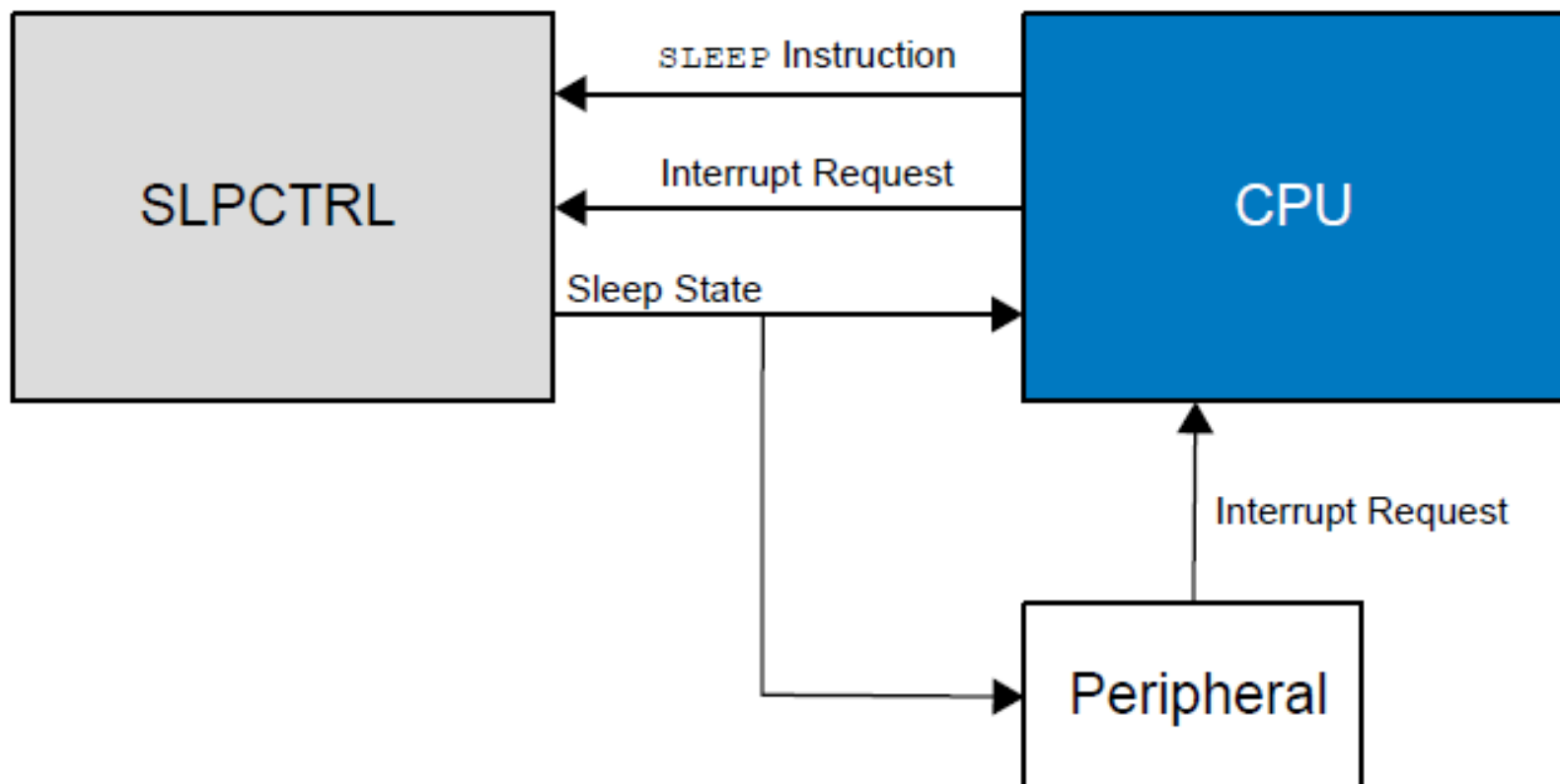


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 22 - Sleep Modes and Power Management



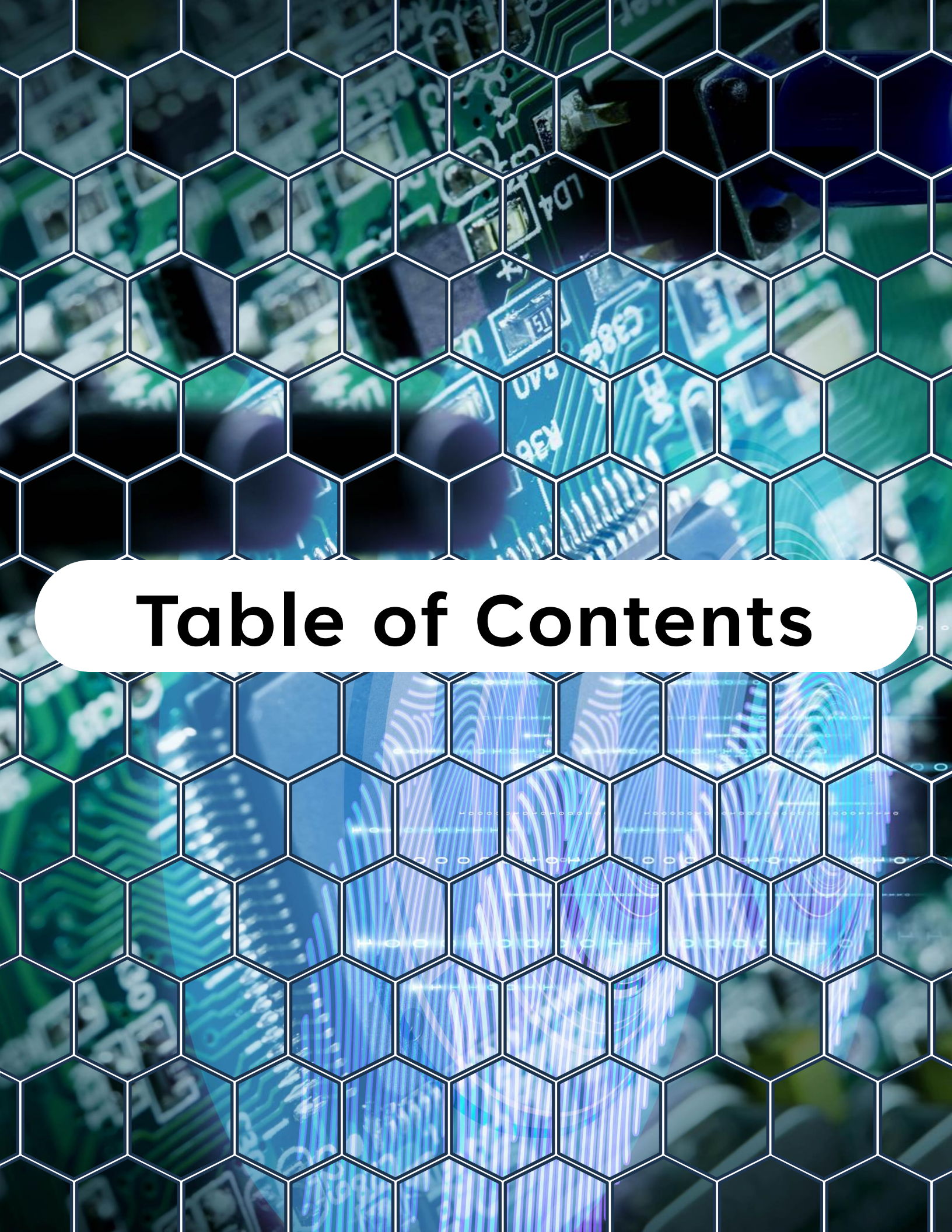
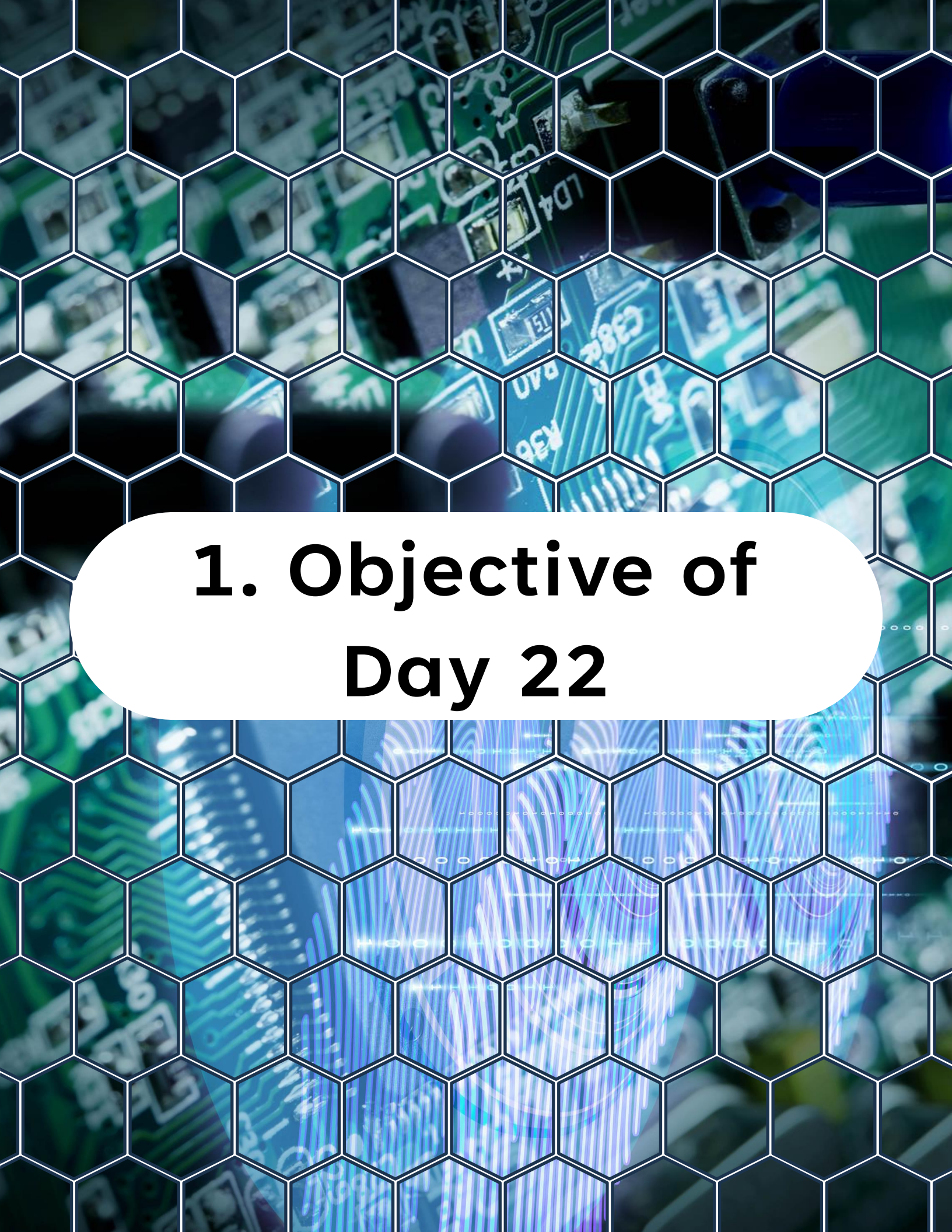


Table of Contents

Table of Contents

1. Objective of Day 22
2. Why Sleep Matters
3. Sleep Modes Overview
4. Entering Sleep Mode
5. Peripheral Wake-Up Sources
6. Power Optimization Techniques
7. Lab - Low-Power Periodic Wake-Up System
8. What Happens Now?
9. Measuring Power Reduction
10. What You Should Understand Now
11. Preview of Day 23



1. Objective of Day 22

1. Objective of Day 22

So far, our firmware has been running continuously. In real embedded systems, that is often unacceptable, power matters.

Today you will learn:

- The sleep modes available on AVR128DA48
- Which peripherals can wake the CPU
- How clock domains affect power consumption
- Practical power optimization techniques
- How to build a low-power periodic wake-up system

This is where embedded systems stop being development boards and start becoming products.



2. Why Sleep Matters

2. Why Sleep Matters

In many applications:

- Sensor nodes
- Battery-powered devices
- Data loggers
- Industrial monitors

The MCU spends most of its time doing nothing.

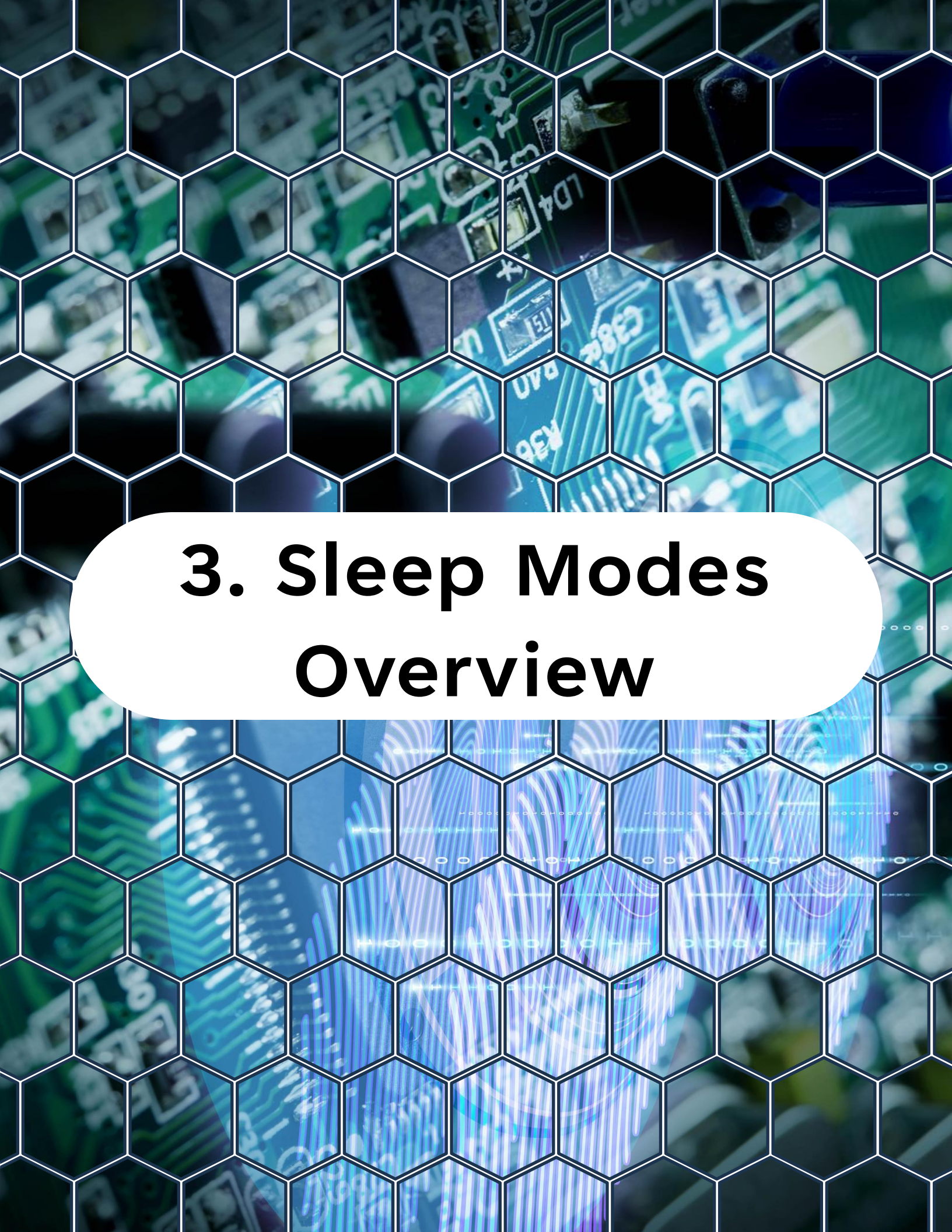
Instead of:

Running a loop forever

We want:

Sleep -> Wake -> Do work -> Sleep again

This drastically reduces current consumption.



3. Sleep Modes Overview

3. Sleep Modes Overview

Sleep modes are used to shut down peripherals and clock domains in the device in order to save power.

In the AVR128DA48 there are four modes available: One Active mode in which software is executed, and three sleep modes.

The available sleep modes are:

- **Idle**
- **Standby**
- **Power-Down** (deepest)

Interrupts are used to wake the device from sleep. The available interrupt wake-up sources depend on the configured sleep mode.

3. Sleep Modes Overview

3.1 Idle Mode

- CPU clock stopped
- Peripherals continue running
- Fast wake-up Good for timer-driven systems

Use when you want timers, ADC, USART, etc. running.

3.2 Standby Mode

- CPU stopped
- Main clock stopped
- Selected peripherals in separate clock domain continue
- Lower power than Idle

Often used with RTC.

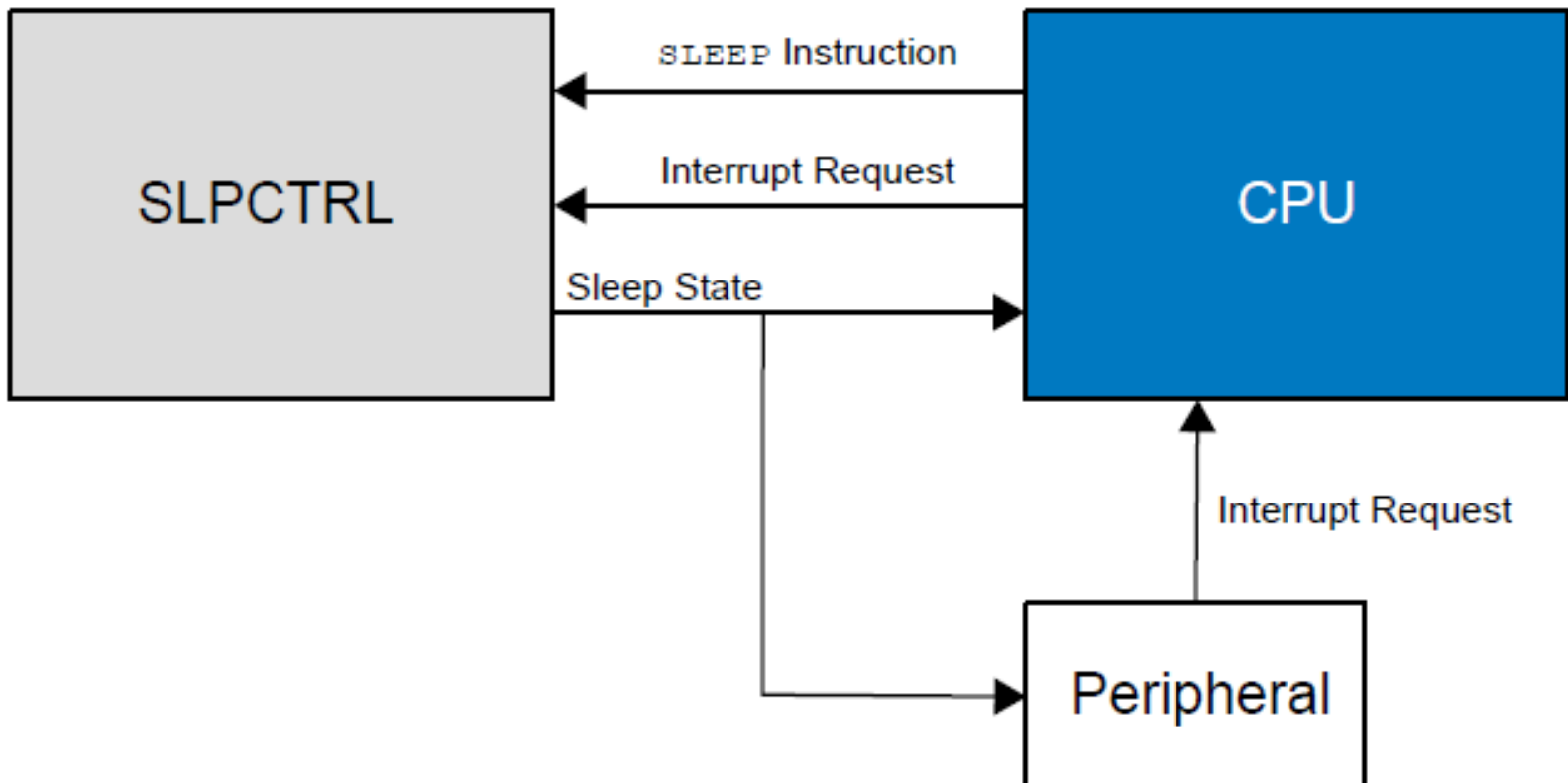
3. Sleep Modes Overview

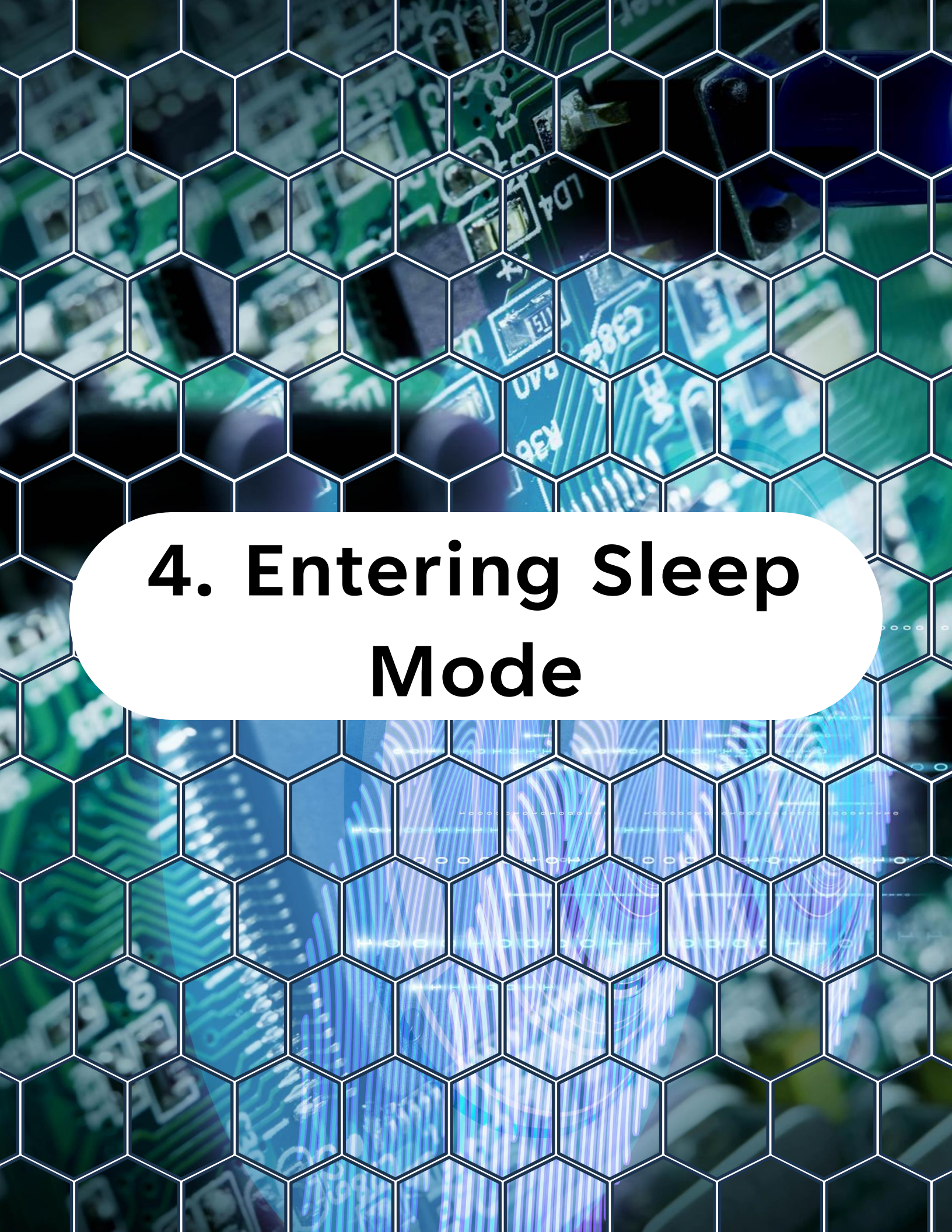
3.3 Power-Down Mode

- Most clocks disabled
- Only asynchronous peripherals active
- Lowest consumption
- Slower wake-up

Best for ultra-low power applications.

Sleep Controller in the System





4. Entering Sleep Mode

4. Entering Sleep Mode

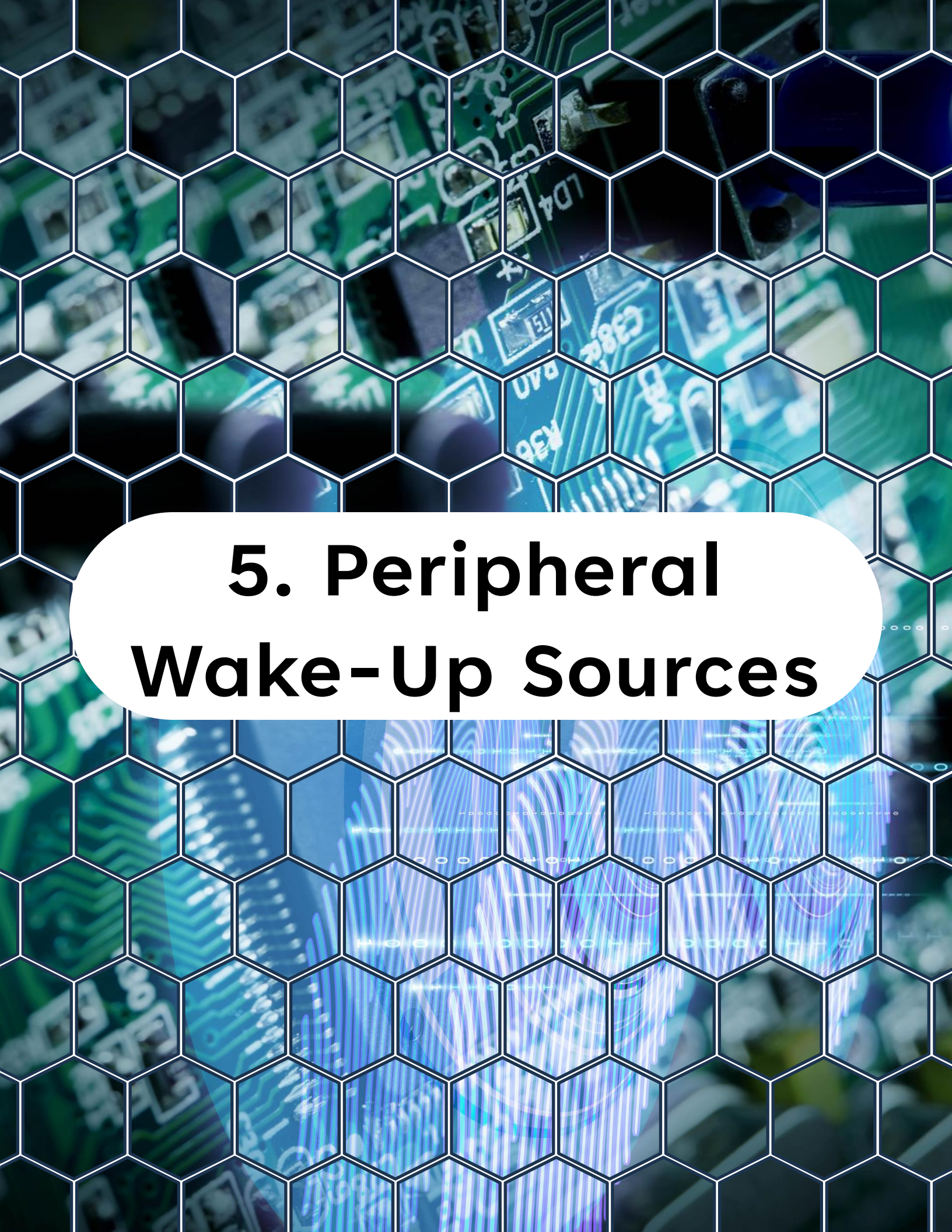
Sleep configuration uses: **SLPCTRL** register.

Basic steps:

1. Select sleep mode
2. Enable sleep
3. Execute sleep instruction

```
1  /**
2   * @brief Enter Idle sleep mode.
3   */
4  static void sleep_idle(void)
5  {
6      // Set sleep mode to Idle and enable sleep.
7      SLPCTRL.CTRLA =
8          SLPCTRL_SMODE_IDLE_gc |
9          SLPCTRL_SEN_bm;
10
11     // Enter sleep mode.
12     __asm__ __volatile__("sleep");
13 }
```

After executing sleep:CPU halts until interrupt occurs.



5. Peripheral Wake-Up Sources

5. Peripheral Wake-Up Sources

A peripheral can wake the CPU if:

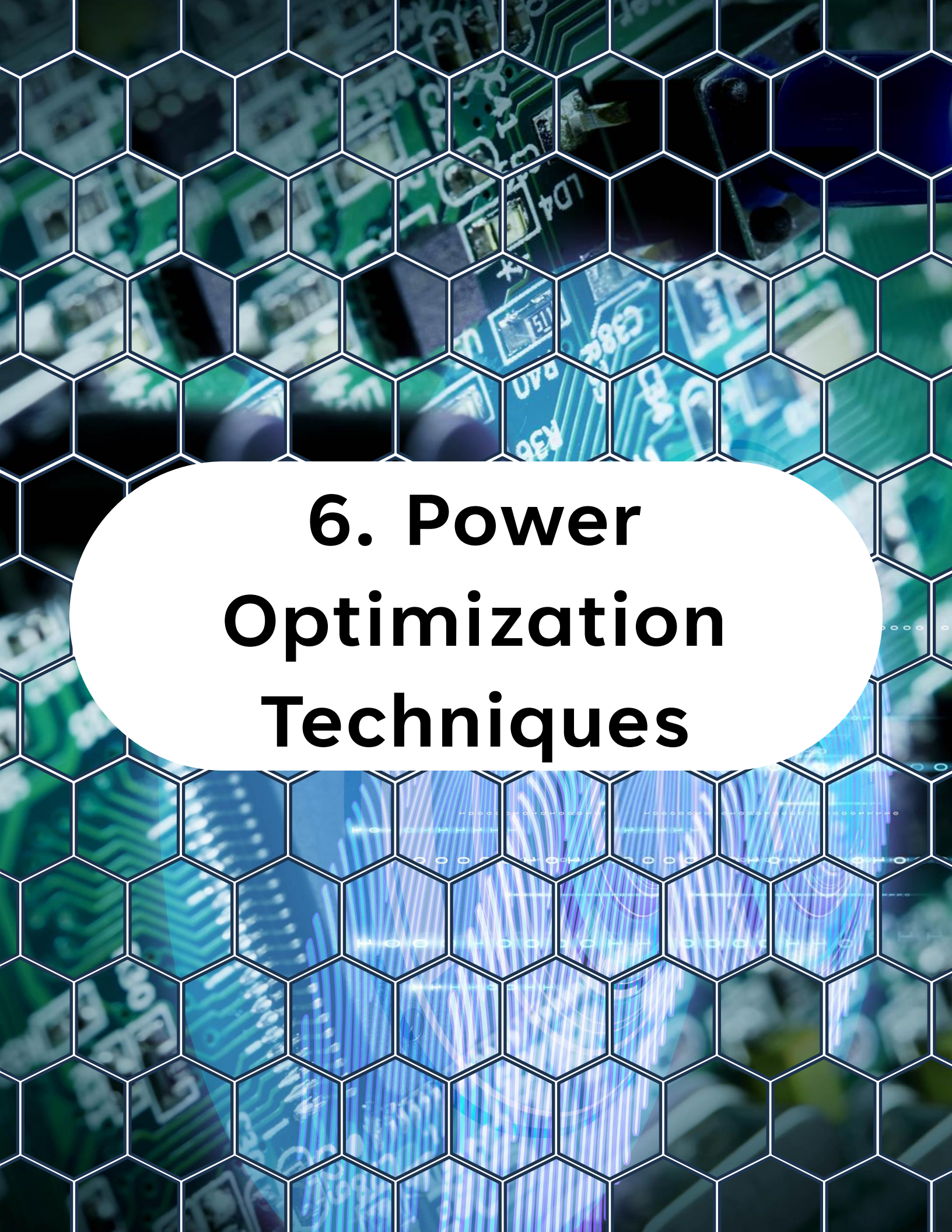
- Its interrupt is enabled
- Global interrupts are enabled

Common wake-up sources:

- RTC overflow
- TCB interrupt
- ADC result ready
- USART RX complete
- External pin interrupt

Important:

If peripheral clock is disabled, it cannot wake the CPU.



6. Power Optimization Techniques

6. Power Optimization Techniques

Reducing power is not only about sleep.

It includes:

6.1 Disable Unused Peripherals

Each peripheral consumes power when clocked.

Disable unused modules:

```
TCA0.SINGLE.CTRLA = 0;
```

```
SPI0.CTRLA = 0;
```

```
USART0.CTRLA = 0;
```

6.2 Reduce System Clock

Lower clock frequency:

- Reduces dynamic power
- Increases execution time

Trade-off must be evaluated.

6. Power Optimization Techniques

6.3 Use Event System

Offload CPU.

Instead of: **Interrupt -> CPU processing**

Use: **Hardware pipeline**

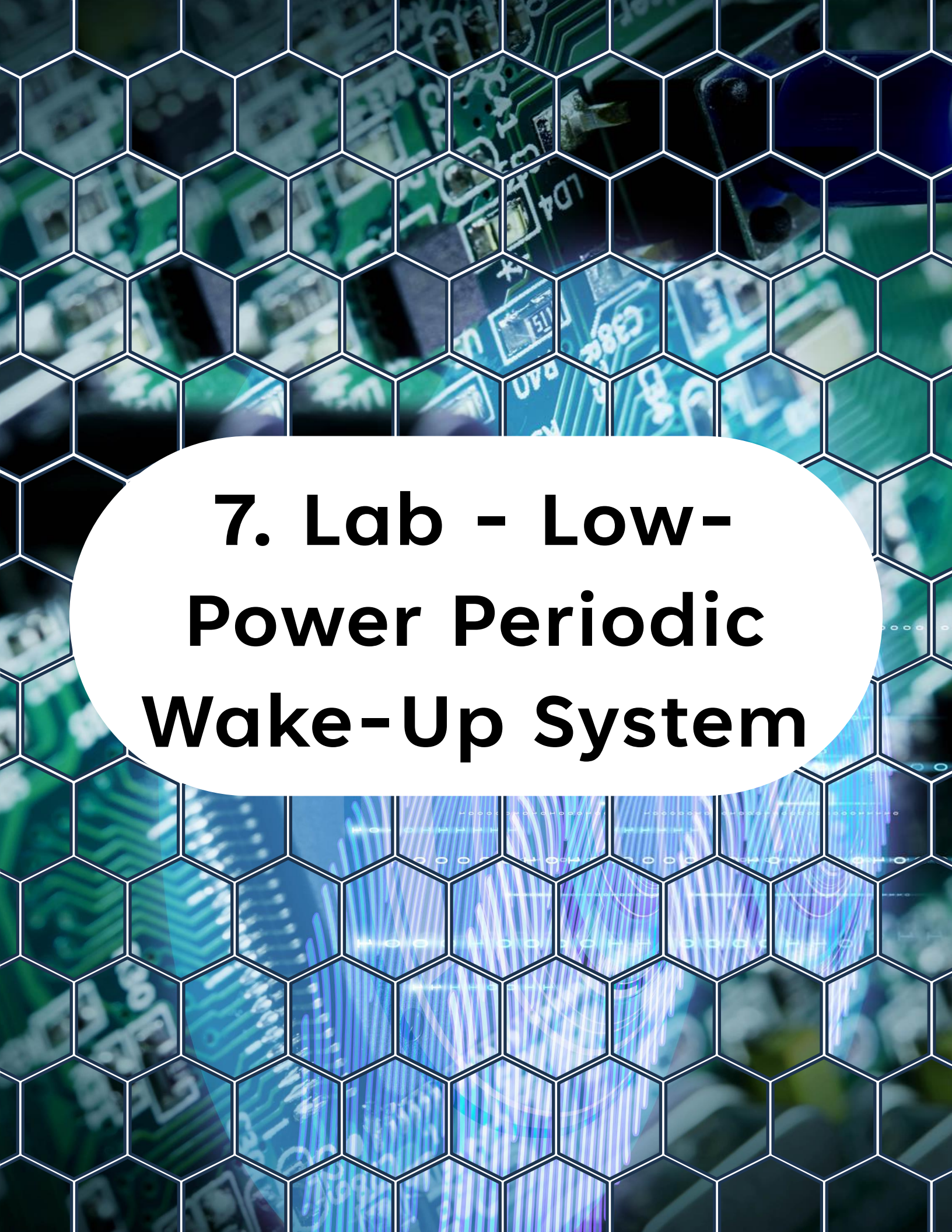
CPU can remain asleep longer.

6.4 Use Standby with RTC

RTC runs from 32 kHz oscillator.

- Low power.
- Precise timing.

Ideal for periodic wake-up.



7. Lab - Low-Power Periodic Wake-Up System

7. Lab - Low-Power Periodic Wake-Up System

Goal

- Use RTC to wake MCU every 1 second
- Toggle LED
- Return to sleep

CPU active only for a few microseconds each second.

7.1 Configure RTC

We use:

- 32.768 kHz internal oscillator
- Periodic interrupt
- 1 second interval

$$32768 \text{ Hz} / 32768 = 1 \text{ second}$$

7. Lab - Low-Power Periodic Wake-Up System

RTC Initialization

```
1  /**
2   * @brief Initialize RTC for 1-second periodic interrupt.
3   */
4  static void RTC_init(void)
5  {
6      // 1. Select internal 32.768 kHz oscillator
7      RTC.CLKSEL = RTC_CLKSEL_OSC32K_gc;
8
9      // 2. Wait for all register to be synchronized
10     while (RTC.STATUS > 0)
11     {
12         ;
13     }
14
15     // 3. Configure PIT: enable,
16     // period = 32 768 RTC cycles = 1 second
17     RTC.PITCTRLA = RTC_PERIOD_CYC32768_gc // 1-second period
18                 | RTC_PITEN_bm;          // Enable PIT
19
20     // 4. Enable PIT interrupt
21     RTC.PITINTCTRL = RTC_PI_bm;
22 }
```


7. Lab - Low-Power Periodic Wake-Up System

7.2 LED Initialization



```
1  #define LED_PORT PORTC
2  // PC6 - LED0 on AVR128DA48 Curiosity Nano
3  #define LED_PIN  PIN6_bm
4
5  /**
6   * @brief Configure PC6 as a push-pull output for LED0.
7   *       LED is active LOW - drive HIGH (off) initially.
8   */
9  static void LED_init(void)
10 {
11     // Set PC6 as output
12     LED_PORT.DIRSET  = LED_PIN;
13
14     // LED off initially (active LOW)
15     LED_PORT.OUTSET  = LED_PIN;
16 }
```

7. Lab - Low-Power Periodic Wake-Up System

7.3 RTC Interrupt



```
1  /**
2   * RTC Periodic Interrupt (fires every 1 second).
3   * Toggle the LED, then return - the CPU goes
4   * straight back to sleep in the main loop.
5   */
6  ISR(RTC_PIT_vect)
7  {
8      /* Clear the PIT interrupt flag */
9      RTC.PITINTFLAGS = RTC_PI_bm;
10
11     /* Toggle LED (PC6, active LOW) */
12     PORTC.OUTTGL = LED_PIN;
13 }
```

7. Lab - Low-Power Periodic Wake-Up System

7.4 Sleep Init

```
1  /**
2   * @brief Select Power-Down as the sleep mode.
3   *       In Power-Down the CPU and most peripherals
4   *       are stopped;
5   *       the RTC continues running from its
6   *       asynchronous clock domain.
7   */
8  static void SLEEP_init(void)
9  {
10     // Power-Down mode and enable Sleep
11     SLPCTRL.CTRLA = SLPCTRL_SMODE_PDOWN_gc
12                   | SLPCTRL_SEN_bm;
13 }
```


7. Lab - Low-Power Periodic Wake-Up System

7.5 Sleep Loop



```
1  #include <avr/io.h>
2  #include <avr/interrupt.h>
3  #include <avr/sleep.h>
4
5  int main(void)
6  {
7      // Configure PC6 as a push-pull output for LED0.
8      LED_init();
9      // Configure the RTC Periodic Interrupt Timer
10     RTC_init();
11     // Select Power-Down as the sleep mode.
12     SLEEP_init();
13     // Enable Global interrupt
14     sei();
15
16     for (;;) {
17         // Enter Power-Down; wake on PIT interrupt
18         sleep_cpu();
19     }
20 }
```



8. What Happens Now?

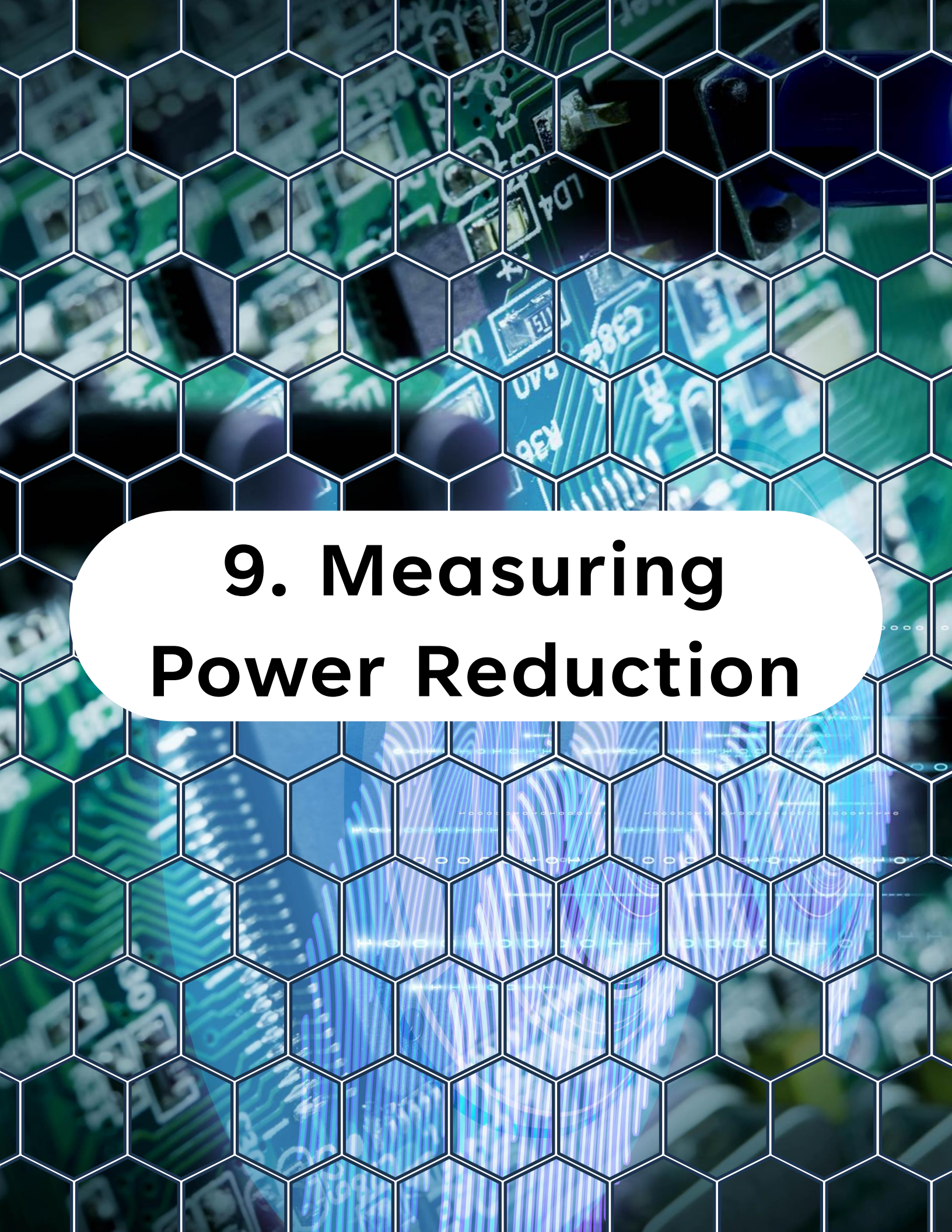
8. What Happens Now?

System behavior:

1. MCU enters Standby mode
2. RTC runs on 32 kHz clock
3. Every 1 second, PIT interrupt occurs
4. CPU wakes
5. ISR toggles LED
6. CPU returns to sleep

Active time per second: extremely small.

Power consumption: drastically reduced.



9. Measuring Power Reduction

9. Measuring Power Reduction

To truly understand power savings:

1. Insert ammeter in series with VDD
2. Measure current in:
 - Active mode
 - Idle mode
 - Standby mode

You will see dramatic differences.

That measurement is part of real embedded engineering.



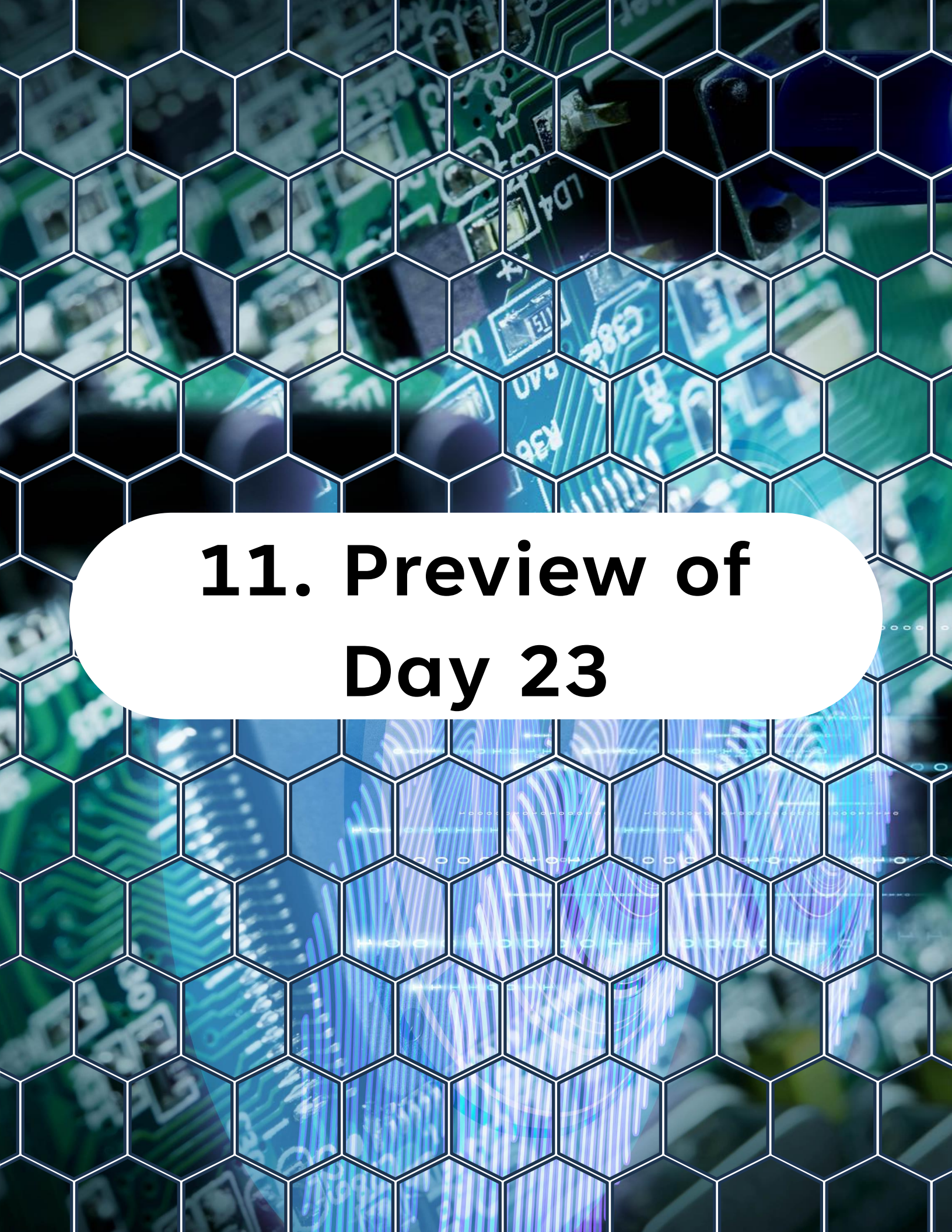
10. What You Should Understand Now

10. What You Should Understand Now

You now know:

- The available sleep modes
- How to enter sleep
- How peripherals wake the CPU
- How to reduce system power
- How to build periodic wake-up systems

You are now designing energy-aware firmware.



11. Preview of Day 23

11. Preview of Day 23

Next:

- Interrupt Architecture
- Interrupt vectors
- Priority handling
- ISR design best practices
- Multi-interrupt systems

You will learn how to build reliable
interrupt-driven systems without chaos.

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

<https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course>